

	UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE Programación Avanzada	NOMBRE	Fernando Sáenz
		Nrc	30405
		FECHA	30/05/2026
INFORME DE RESULTADOS			

Tema

AUTÓNOMO Comparar la eficiencia y legibilidad de distintos paradigmas de programación mediante la resolución de un problema algorítmico complejo.

1. Contexto de la Problemática

Has sido contratado para desarrollar un nuevo módulo en el ecosistema **ESPE-Tech**. La institución requiere un sistema que gestione el inventario de equipos tecnológicos de los laboratorios. Basado en la estructura del proyecto *ESPE-Plus*, deberás implementar una solución que procese grandes volúmenes de datos de equipos para generar reportes analíticos de disponibilidad y valoración.

2. Instrucciones Técnicas

Deberás crear un nuevo proyecto Spring Boot siguiendo la arquitectura evidenciada en clases:

- **Capa de Persistencia:** Definir una entidad `HardwareEntity` con atributos como: id, modelo, categoria (Laptop, PC, Servidor), precio (BigDecimal), fechaCompra y estado (ACTIVO, DEBAJA).
- **Capa de Negocio (Service):** Implementar la lógica de procesamiento algorítmico solicitado.
- **Capa Web (Controller):** Exponer los resultados mediante un endpoint REST.
- **Capa AI:** Integrar `AIService` para generar un mensaje dinámico de "Resumen de Inventario" (similar a `ESPEPlusAIService`).

3. El Problema Algorítmico:

Dado un conjunto de 10,000 registros de hardware, el sistema debe:

1. Filtrar solo equipos comprados en los últimos 5 años y que estén en estado 'ACTIVO'.
2. Agrupar los equipos por categoría.
3. Calcular el valor total y el promedio de precio por cada categoría.
4. Retornar el equipo más caro de cada categoría.

4. Descripción de la Tarea

El estudiante debe implementar el problema algorítmico anterior mediante dos enfoques distintos dentro del mismo servicio:

Paradigma Imperativo: Uso estricto de estructuras de control tradicionales (for, if, else, acumuladores manuales).

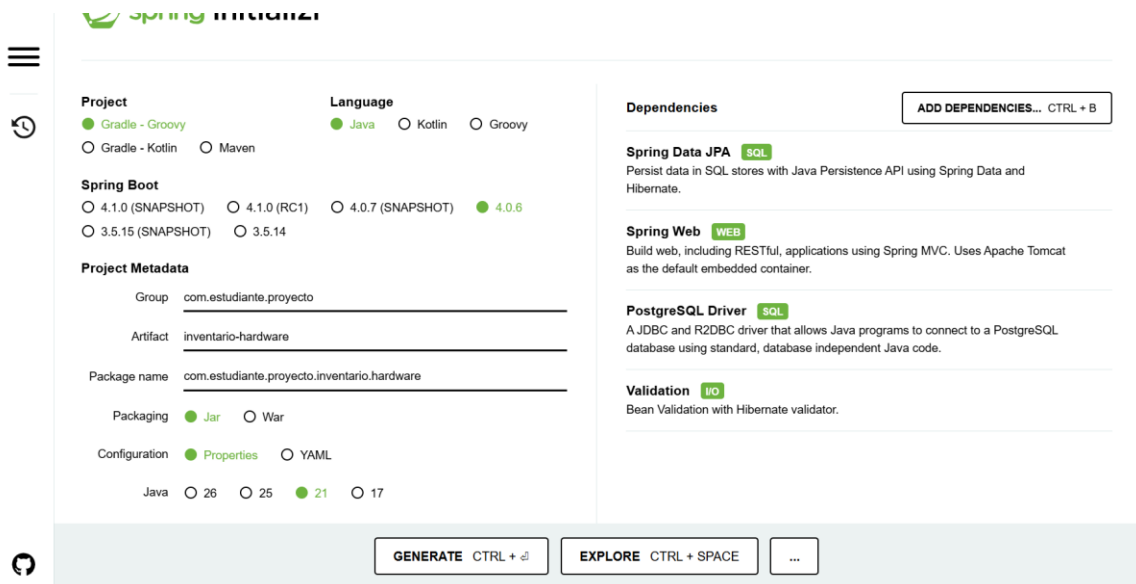
Paradigma Funcional/Declarativo: Uso de Java Streams API, Optional y Collectors.

Finalice subiendo el link de su repositorio de GitHub y realizando un breve informe en el README del proyecto, comparando ambos enfoques en términos de líneas de código, legibilidad y facilidad de mantenimiento.

	UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE Programación Avanzada	NOMBRE	Fernando Sáenz
		Nrc	30405
		FECHA	30/05/2026
	INFORME DE RESULTADOS		

5. Desarrollo

Paso 1. Creación del proyecto Spring Boot



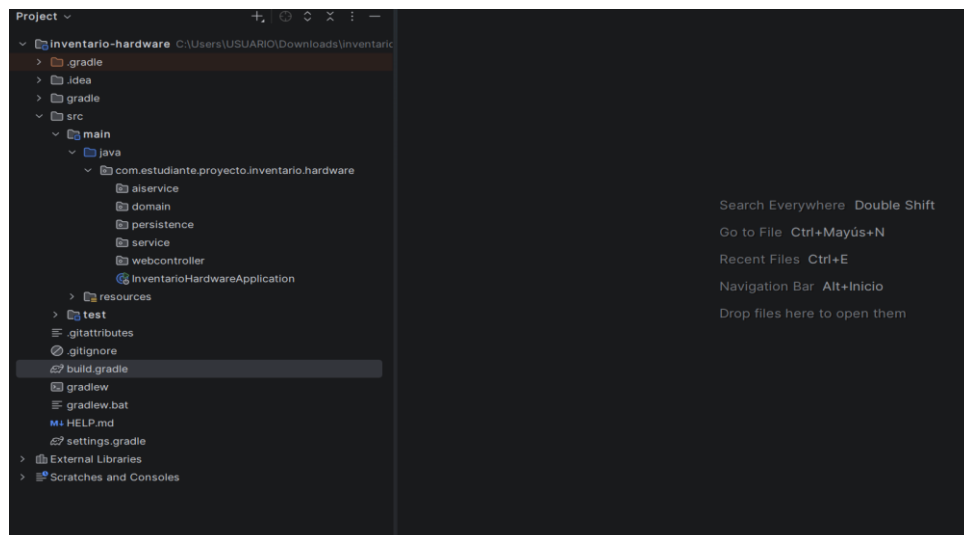
The screenshot shows the Spring Initializr web interface. On the left, there are navigation icons. The main area is divided into sections: Project, Language, Spring Boot, Project Metadata, and Dependencies.

- Project:** Gradle - Groovy (selected), Gradle - Kotlin, Maven.
- Language:** Java (selected), Kotlin, Groovy.
- Spring Boot:** 4.1.0 (SNAPSHOT), 4.1.0 (RC1), 4.0.7 (SNAPSHOT), 4.0.6 (selected), 3.5.15 (SNAPSHOT), 3.5.14.
- Project Metadata:**
 - Group: com.estudiante.proyecto
 - Artifact: inventario-hardware
 - Package name: com.estudiante.proyecto.inventario.hardware
 - Packaging: Jar (selected), War.
 - Configuration: Properties (selected), YAML.
 - Java: 26, 25, 21 (selected), 17.
- Dependencies:**
 - Spring Data JPA (SQL): Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
 - Spring Web (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
 - PostgreSQL Driver (SQL): A JDBC and R2DBC driver that allows Java programs to connect to a PostgreSQL database using standard, database independent Java code.
 - Validation (JSR): Bean Validation with Hibernate validator.

At the bottom, there are buttons: GENERATE (CTRL + G), EXPLORE (CTRL + SPACE), and a menu icon.

Para el desarrollo de la práctica se creó un nuevo proyecto Spring Boot utilizando la plataforma Spring Initializr. Se seleccionó Gradle como herramienta de construcción, Java como lenguaje de programación y Spring Boot versión 4.0.6. Adicionalmente, se configuró el paquete principal `com.estudiante.proyecto.inventario.hardware` y se incorporaron las dependencias necesarias para el desarrollo de la aplicación, entre ellas Spring Web, Spring Data JPA, PostgreSQL Driver y Validation. Esta configuración proporciona la base necesaria para implementar la arquitectura por capas y desarrollar los servicios requeridos por la aplicación.

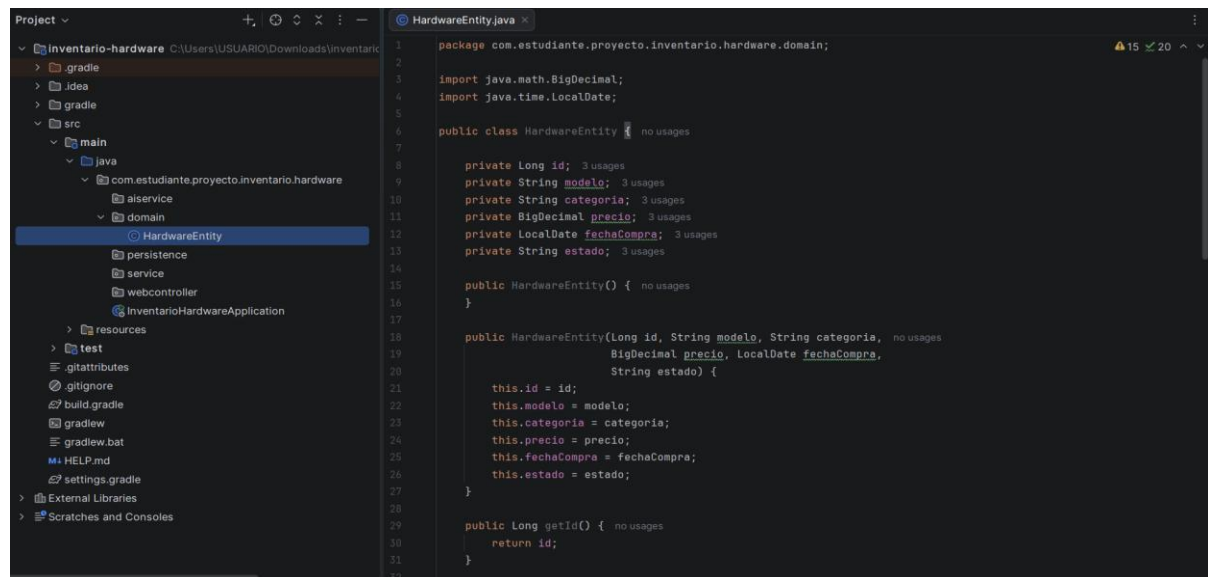
Paso 2: Creación de paquetes



	UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE Programación Avanzada	NOMBRE	Fernando Sáenz
		Nrc	30405
		FECHA	30/05/2026
	INFORME DE RESULTADOS		

En este paso se organizó el proyecto utilizando una arquitectura por capas. Se crearon los paquetes domain, persistence, service, webcontroller y aIService, con la finalidad de separar responsabilidades dentro del sistema. Esta estructura permite mantener un código más ordenado, facilitar el mantenimiento y diferenciar la lógica de negocio, la persistencia de datos, los controladores REST y el servicio relacionado con la generación del resumen mediante IA.

Paso 3: Crear la entidad HardwareEntity



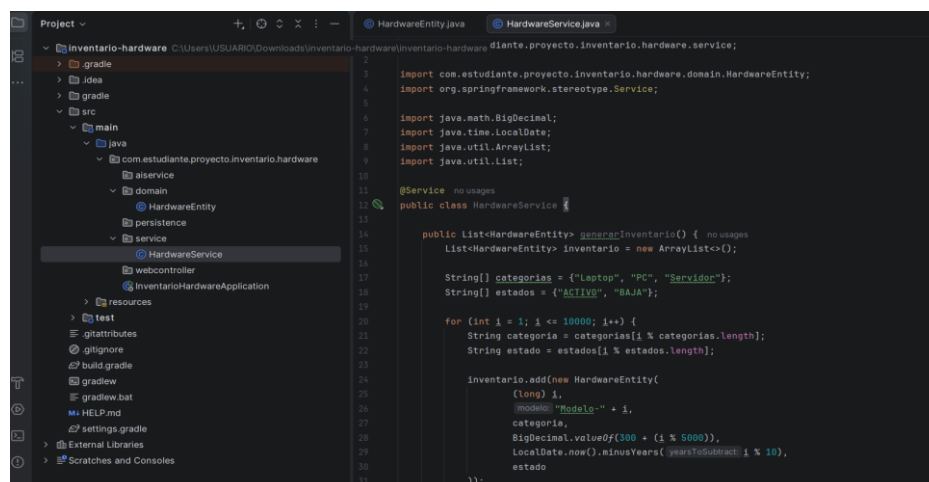
```

1 package com.estudiante.proyecto.inventario.hardware.domain;
2
3 import java.math.BigDecimal;
4 import java.time.LocalDate;
5
6 public class HardwareEntity {
7
8     private Long id;
9     private String modelo;
10    private String categoria;
11    private BigDecimal precio;
12    private LocalDate fechaCompra;
13    private String estado;
14
15    public HardwareEntity() {
16    }
17
18    public HardwareEntity(Long id, String modelo, String categoria,
19                          BigDecimal precio, LocalDate fechaCompra,
20                          String estado) {
21        this.id = id;
22        this.modelo = modelo;
23        this.categoria = categoria;
24        this.precio = precio;
25        this.fechaCompra = fechaCompra;
26        this.estado = estado;
27    }
28
29    public Long getId() {
30        return id;
31    }
32

```

Se creó la entidad HardwareEntity dentro de la capa Domain para representar los equipos tecnológicos del inventario. La clase contiene los atributos id, modelo, categoría, precio, fecha de compra y estado, además de sus respectivos constructores, métodos getters y setters. Esta entidad servirá como base para el procesamiento y gestión de la información utilizada por los servicios y controladores de la aplicación.

Paso 4: Crear HardwareService



```

1 package com.estudiante.proyecto.inventario.hardware.service;
2
3 import com.estudiante.proyecto.inventario.hardware.domain.HardwareEntity;
4 import org.springframework.stereotype.Service;
5
6 import java.math.BigDecimal;
7 import java.time.LocalDate;
8 import java.util.ArrayList;
9 import java.util.List;
10
11 @Service
12 public class HardwareService {
13
14    public List<HardwareEntity> generarInventario() {
15        List<HardwareEntity> inventario = new ArrayList<>();
16
17        String[] categorias = {"Laptop", "PC", "Servidor"};
18        String[] estados = {"Activo", "Baja"};
19
20        for (int i = 1; i <= 1000; i++) {
21            String categoria = categorias[i % categorias.length];
22            String estado = estados[i % estados.length];
23
24            inventario.add(new HardwareEntity(
25                (long) i,
26                "Modelo-" + i,
27                categoria,
28                BigDecimal.valueOf(300 + (i % 500)),
29                LocalDate.now().minusYears((yearsToSubtract i % 10)),
30                estado
31            ));
32        }
33    }
34

```

	UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE Programación Avanzada	NOMBRE	Fernando Sáenz
		Nrc	30405
		FECHA	30/05/2026
	INFORME DE RESULTADOS		

En este paso se creó la clase `HardwareService` dentro de la capa de servicio. Esta clase permite generar una lista de 10.000 registros simulados de hardware, utilizando diferentes categorías como Laptop, PC y Servidor, además de estados como ACTIVO y BAJA. Esta información servirá como base para aplicar posteriormente los enfoques imperativo y funcional solicitados en la actividad.

Paso 5: Crear el procesamiento algorítmico con los dos enfoques: imperativo y funcional.

```

public class HardwareService {
    public List<HardwareEntity> generarInventario() { 2 usages
        return inventario;
    }

    public List<HardwareEntity> procesarImperativo() { no usages
        List<HardwareEntity> inventario = generarInventario();
        List<HardwareEntity> resultado = new ArrayList<>();

        for (HardwareEntity hardware : inventario) {
            if ("ACTIVO".equals(hardware.getEstado())
                && hardware.getFechaCompra().isAfter(LocalDate.now().minusYears(5))) {
                resultado.add(hardware);
            }
        }

        return resultado;
    }

    public List<HardwareEntity> procesarFuncional() { no usages
        return generarInventario().stream()
            .filter(hardware -> "ACTIVO".equals(hardware.getEstado()))
            .filter(hardware -> hardware.getFechaCompra()
                .isAfter(LocalDate.now().minusYears(5)))
            .toList();
    }
}

```

En este paso se implementaron dos métodos de procesamiento dentro de la clase `HardwareService`. El primer método utiliza el enfoque imperativo mediante estructuras tradicionales como ciclos `for` y condicionales `if`. El segundo método aplica el enfoque funcional utilizando la API Stream de Java. Ambos métodos filtran los equipos con menos de cinco años de uso y que se encuentran en estado ACTIVO.

Paso 6: Crear el controlador REST.

```

package com.estudiante.proyecto.inventario.hardware.webcontroller;

import com.estudiante.proyecto.inventario.hardware.domain.HardwareEntity;
import com.estudiante.proyecto.inventario.hardware.service.HardwareService;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class HardwareController {

    private final HardwareService hardwareService; 3 usages

    public HardwareController(HardwareService hardwareService) {
        this.hardwareService = hardwareService;
    }

    @GetMapping("/imperativo")
    public List<HardwareEntity> imperativo() {
        return hardwareService.procesarImperativo();
    }

    @GetMapping("/funcional")
    public List<HardwareEntity> funcional() {
        return hardwareService.procesarFuncional();
    }
}

```

	UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE Programación Avanzada	NOMBRE	Fernando Sáenz
		Nrc	30405
		FECHA	30/05/2026
	INFORME DE RESULTADOS		

Se creó el controlador HardwareController dentro de la capa web, con el objetivo de exponer los resultados del procesamiento mediante endpoints REST. Se implementaron dos rutas: una para ejecutar el enfoque imperativo y otra para el enfoque funcional, permitiendo visualizar los datos procesados desde el navegador o herramientas como Postman.

Paso 7: Pruebas de los dos endpoints.

- Prueba del enfoque imperativo

```
[
  {
    "id": 2,
    "modelo": "Modelo-2",
    "categoria": "Servidor",
    "precio": 302,
    "fechaCompra": "2024-05-30",
    "estado": "ACTIVO"
  },
  {
    "id": 4,
    "modelo": "Modelo-4",
    "categoria": "PC",
    "precio": 304,
    "fechaCompra": "2022-05-30",
    "estado": "ACTIVO"
  },
  {
    "id": 10,
    "modelo": "Modelo-10",
    "categoria": "PC",
    "precio": 310,
    "fechaCompra": "2026-05-30",
    "estado": "ACTIVO"
  },
  {
    "id": 12,
    "modelo": "Modelo-12",
    "categoria": "Laptop",
    "precio": 312,
    "fechaCompra": "2024-05-30",
    "estado": "ACTIVO"
  },
  {
    "id": 14,
    "modelo": "Modelo-14",
    "categoria": "Servidor",
    "precio": 314,
    "fechaCompra": "2022-05-30",
    "estado": "ACTIVO"
  }
]
```

Se ejecutó el endpoint /imperativo para verificar el procesamiento de los datos mediante estructuras de control tradicionales. El sistema filtró correctamente los equipos que cumplen las condiciones establecidas, mostrando los resultados en formato JSON.

	UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE Programación Avanzada	NOMBRE	Fernando Sáenz
		Nrc	30405
		FECHA	30/05/2026
	INFORME DE RESULTADOS		

- **Prueba del enfoque funcional**

```
[
  {
    "id": 2,
    "modelo": "Modelo-2",
    "categoria": "Servidor",
    "precio": 302,
    "fechaCompra": "2024-05-30",
    "estado": "ACTIVO"
  },
  {
    "id": 4,
    "modelo": "Modelo-4",
    "categoria": "PC",
    "precio": 304,
    "fechaCompra": "2022-05-30",
    "estado": "ACTIVO"
  },
  {
    "id": 10,
    "modelo": "Modelo-10",
    "categoria": "PC",
    "precio": 310,
    "fechaCompra": "2026-05-30",
    "estado": "ACTIVO"
  },
  {
    "id": 12,
    "modelo": "Modelo-12",
    "categoria": "Laptop",
    "precio": 312,
    "fechaCompra": "2024-05-30",
    "estado": "ACTIVO"
  },
  {
    "id": 14,
    "modelo": "Modelo-14",
    "categoria": "Servidor",
    "precio": 314,
    "fechaCompra": "2022-05-30",
    "estado": "ACTIVO"
  },
  {
    "id": 20,
    "modelo": "Modelo-20",
    "categoria": "Servidor",
    "precio": 320,
    "estado": "ACTIVO"
  }
]
```

Se ejecutó el endpoint /funcional, implementado mediante Streams de Java. Los resultados obtenidos fueron equivalentes a los del enfoque imperativo, demostrando que ambos paradigmas permiten resolver el mismo problema de manera correcta.

Se demuestra que los dos endpoints están funcionando correctamente.

- <http://localhost:8080/imperativo>
- <http://localhost:8080/funcional>

6. Comparación de resultados

Al comparar las respuestas de ambos endpoints se observó que los registros devueltos son

	UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE Programación Avanzada	NOMBRE	Fernando Sáenz
		Nrc	30405
		FECHA	30/05/2026
	INFORME DE RESULTADOS		

iguales, ya que aplican los mismos criterios de filtrado. La principal diferencia radica en la forma de implementación del código, donde el enfoque imperativo utiliza estructuras de control tradicionales y el funcional emplea operaciones de Streams para realizar el procesamiento.

7. Conclusiones

Durante el desarrollo de la práctica se implementaron dos paradigmas de programación para resolver el mismo problema de negocio. Se comprobó que ambos generan los mismos resultados, pero el enfoque funcional ofrece un código más compacto y fácil de mantener, mientras que el imperativo permite visualizar de forma más explícita cada paso del proceso.

8. Recomendaciones

1. Mantener una estructura organizada del proyecto utilizando paquetes separados para las capas de dominio, servicio y controlador, facilitando el mantenimiento del código.
2. Realizar pruebas periódicas de los endpoints para verificar que los resultados obtenidos sean correctos en ambos paradigmas de programación.
3. Utilizar programación funcional cuando se trabajen grandes colecciones de datos, ya que permite escribir código más limpio y legible.
4. Documentar adecuadamente los métodos y clases del proyecto para facilitar futuras modificaciones o ampliaciones del sistema.
5. Implementar mecanismos de validación y manejo de excepciones para mejorar la robustez y confiabilidad de la aplicación.

9. Bibliografía

- Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley Professional.
- Deitel, P., & Deitel, H. (2020). *Java: How to Program* (12.ª ed.). Pearson.
- Oracle. (2025). *Java Documentation*. Recuperado de: [Oracle Java Documentation](#)
- [Spring Boot Documentation](#). (2025). *Spring Boot Reference Documentation*.
- [Spring Framework Documentation](#). (2025). *Spring Framework Documentation*.

Nombre	Firma	Fecha
Elaborado: Fernando Saenz Estudiante		2026-05-30